

CSE 416: Software Design

Minsoo Ahn
Sungmin Choo
Dahyun Kwon

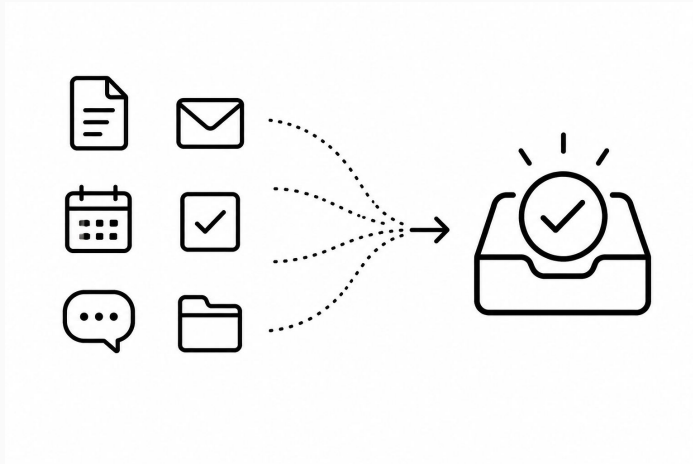
April 29, 2026

Table of Contents

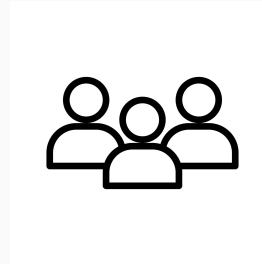
Introduction	1
Requirements	2
System Architecture	3
- Overview / Data Design	4
- UML Sequence Diagrams	5
- API Design / Deployment	6
- Code Conventions	7
Schedule	8
Q&A	9

I . Summary of Problem & Solution

Product Description



Scope & Target Users



I . Summary of Problem & Solution

Feedback



**Personalized
just for you!**

Existing Alternatives



 rosebud

⇒ requires manual effort:(

Definitions

- **Custom Briefing**
- **Macro Events**
- **Diary Cards**
- **Midnight Synthesis**

II. Requirements

Functional

- **Briefing & Widgets:** Dynamic synthesis of Weather, Stocks, and News via Groq LLM.
- **Journaling & Daily Questions:** Automated midnight diary generation and interactive "Diary Cards."
- **Organization:** Two-way Google Calendar sync and integrated To-Do management.
- **User Persona:** Adaptive interest tracking using the **30-day Keyword Decay Formula**.

$$Score = \sum_i w_i \cdot \frac{30 - d_i}{30}, \quad d_i < 30$$

$w_i = 2$ (personal Q&A keyword), $w_i = 1$ (diary keyword)

II. Requirements

Use Cases: Morning Routine

UC-01: Read the Morning Briefing	
Actor	Registered User
Priority	Essential
Scenario	1. The user opens the Morning Briefing.AI dashboard in a browser. 2. If it is the first opening time since midnight and the modal is active, the system displays the full AI briefing modal (10 to 15 lines). 3. Users read the briefing, including weather, market data, major news, and today's schedule. 4. The user closes the modal and checks the 3-line summary in BriefingWidget. 5. The user scrolls through the middle column for more information from WeatherWidget, StocksWidget, and TrendsWidget.
Extensions	• 2a. If API calls fail: cached data or mock data is displayed. Briefing continues to be generated using available data. • 2b. If the user disables automatic modal in Settings: Modal does not appear. The user reads manually by clicking on the BriefingWidget detail icon.

II. Requirements

Use Cases: Daily Questions

UC-02: Respond to the Daily Journaling Question

Actor	Registered User
Priority	Essential
Scenario	1. The user sees the DiaryCard with today's reflection questions in the left column. 2. Users click on the text field and type their answers. 3. The system automatically saves the answer to the Supabase on blur 4. Users can optionally add notes to the Memo field. 5. At midnight (00:00 a.m.), the system includes both the response and memo in the [Personal Records] section of the journal.
Extensions	• 3a. If the Supabase write fails: the system retries up to 3 times, then saves to localStorage with a non-blocking error notice. • If the user has not answered before 00:00, the journal entry notes 'No daily reflection recorded.'

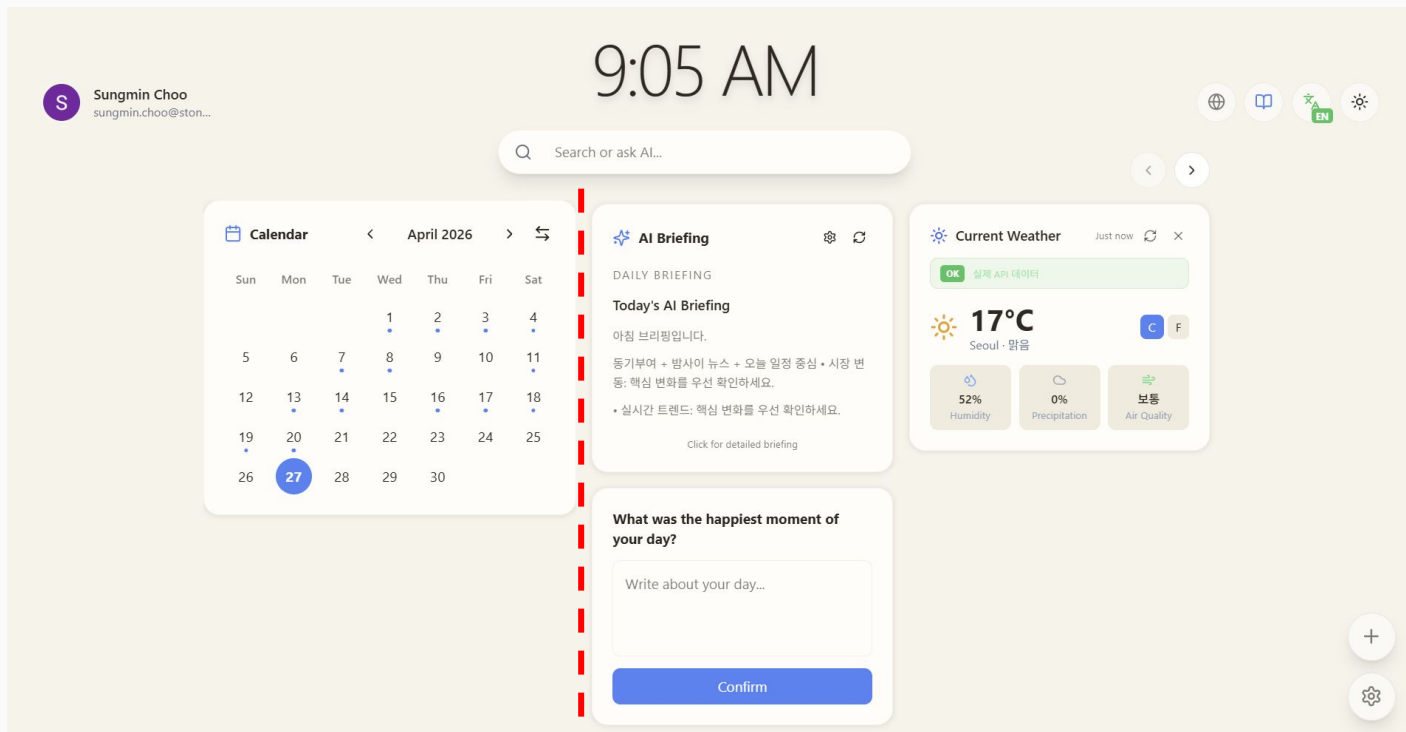
II. Requirements

Use Cases: Midnight Archiving

UC-03: Automatic Midnight Journal Generation (System)	
Actor	System (Automated Process)
Priority	Essential
Scenario	1. At 00:00 the system starts the Midnight Synthesis process. 2. The system collects all the checked and completed calendar events in the last 24 hours. 3. The system searches for responses and notes to today's diary questions. 4. The system takes a snapshot of macro data (weather, stocks, and major news). 5. The system sends structured context payloads to Groq LLM and requests fact-based journal entries. 6. Groq returns a journal (approximately 300 to 500 words) that contains the [Macro Events] and [Personal Records] sections. 7. The system displays preview modals. Users can review, edit, or confirm saves. 8. Items are saved in the Supabase as 'completed' or 'user_edited'.
Extensions	• 1a. If a user's browser closes at 00:00: Synthesis runs the next time a session opens. Modals appear at dashboard loading. • 5a. If Groq times out (>30s): the system will try again. If it still fails, the raw data will be saved as a draft and the user will be notified. • 7a. When the user closes the modal without confirmation: the input will be saved automatically after 30 seconds.

II. Requirements

User Interfaces

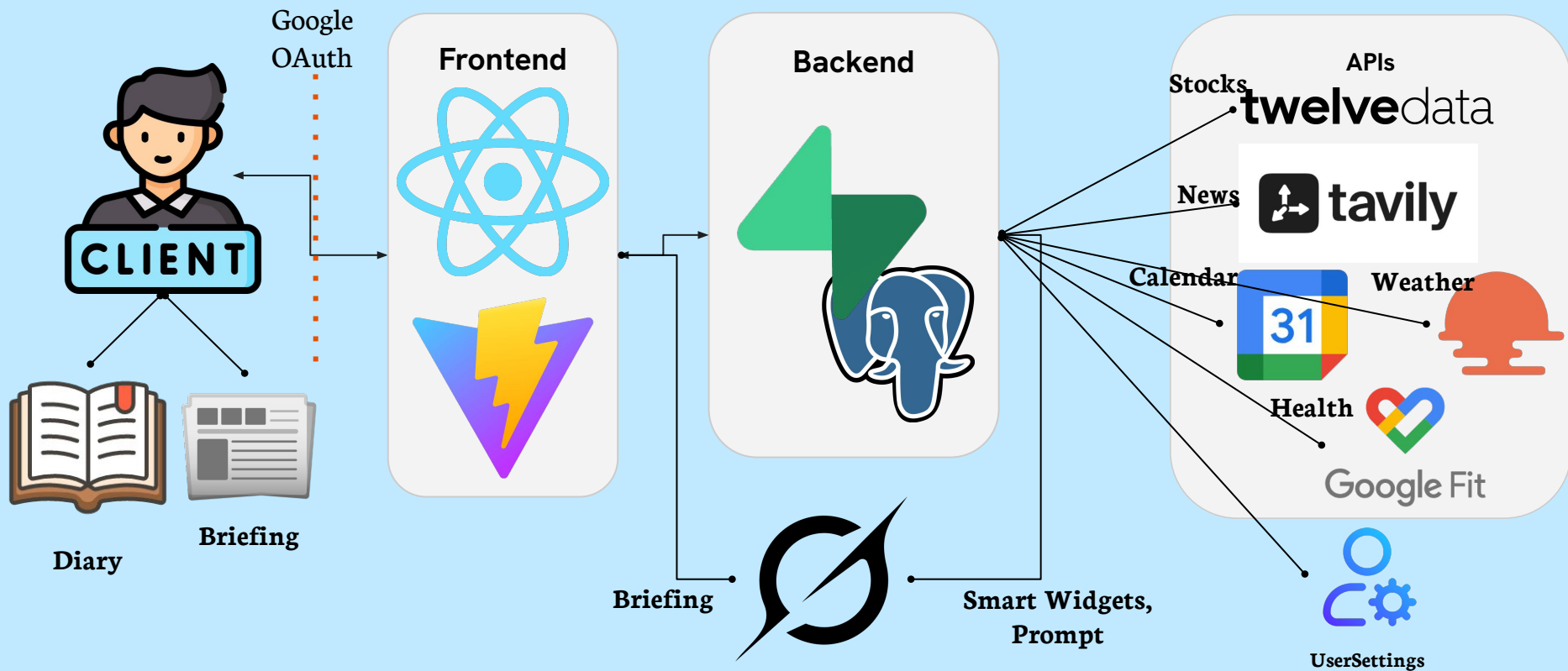


II. Requirements

Non-Functional

- **Performance:** < 3-second initial load via ApiCache; < 10-second AI generation.
- **Security:** Row Level Security (RLS) for total data isolation; server-side secret management.
- **Compatibility:** Optimized for Desktop browsers (Chrome/Edge) with width > 1200px.

III. System Architecture - Overview



III. System Architecture - Overview

System Architecture Diagram:

[Frontend: React/Vite] ↔ [Backend: Supabase Edge Functions] ↔ [Database: PostgreSQL]

[External APIs: Groq, Tavily, Google(Calendar, Fitness), Twelve Data]

Core Technology Stack:

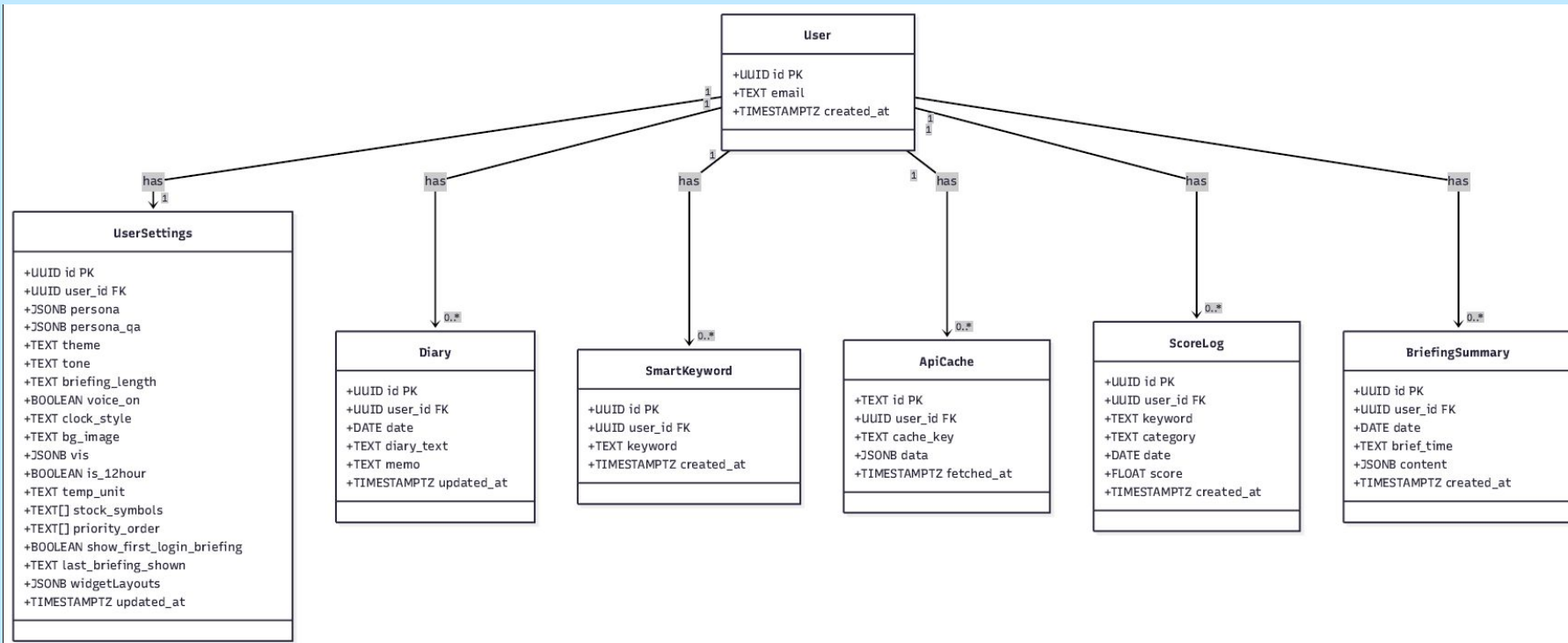
- Frontend: React 18, Vite 6, Tailwind CSS 3
- Backend & Infra: Supabase (PostgreSQL, Auth), Edge Functions (TypeScript)
- State Management: Zustand (Store-based architecture)

Third-Party Components:

- AI & Search: Groq (LLM), Tavily (AI-optimized News/Trends)
- Productivity: Google Calendar & Fitness API
- UI/UX: Framer Motion (Animations), @hello-pangea/dnd (Drag & Drop)

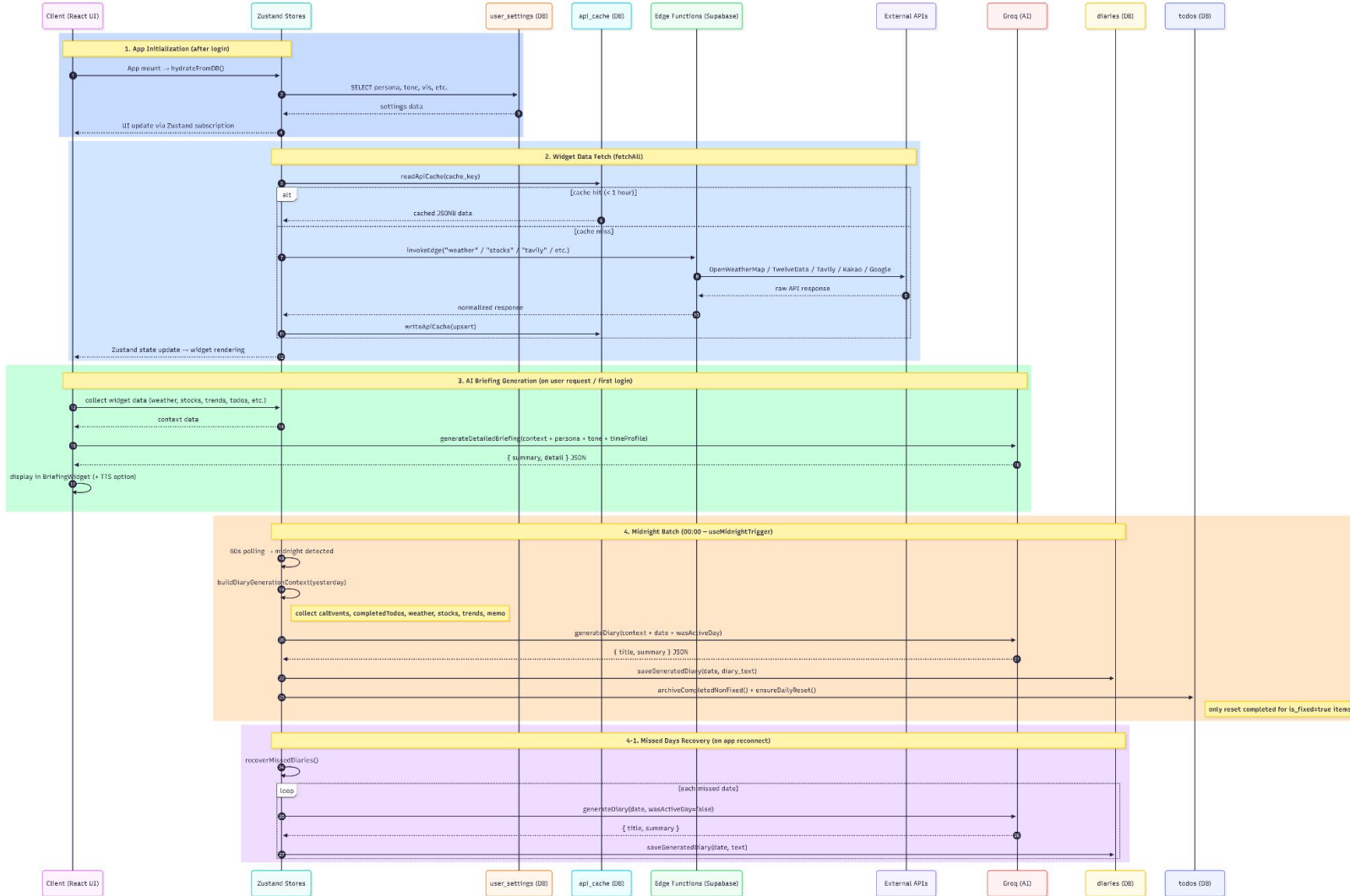
Design Pattern: Component-Based Architecture

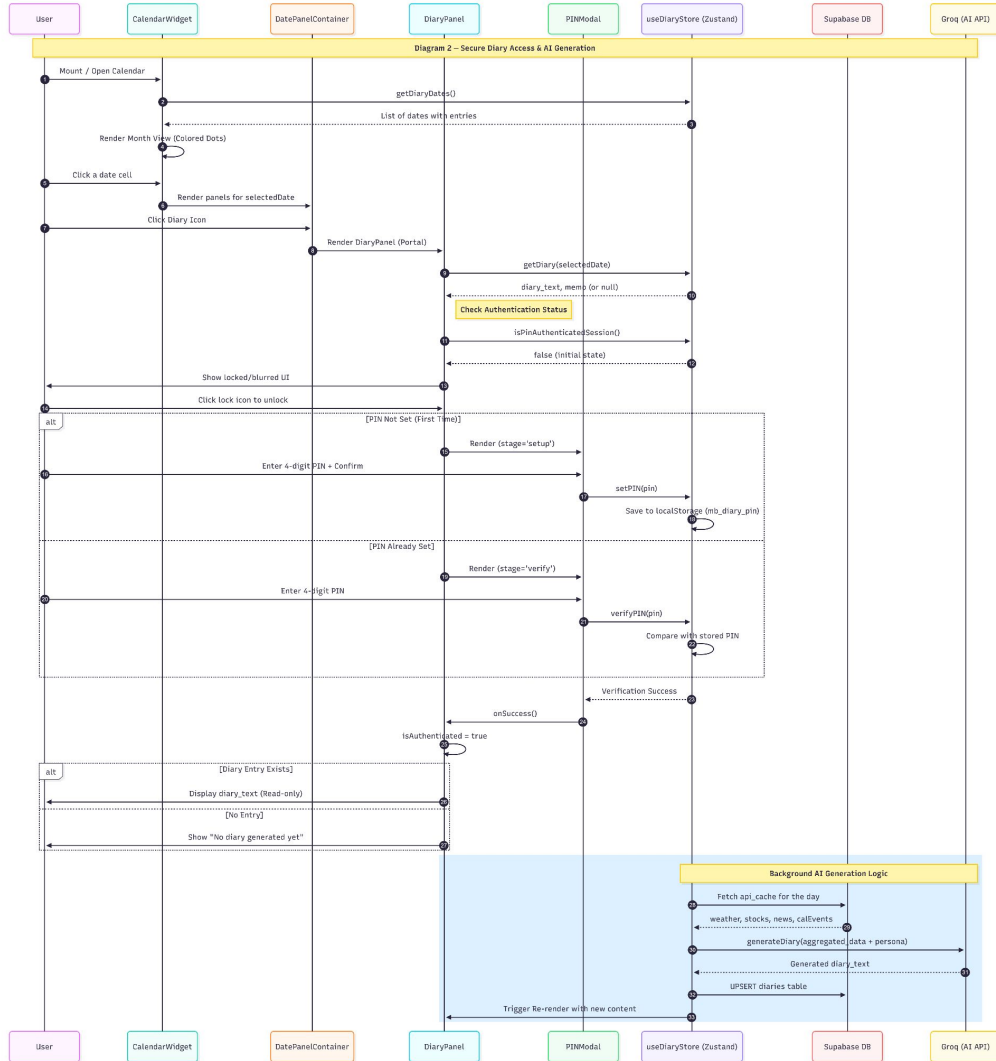
III. System Architecture - Data Design

[link](#)

III. System Architecture UML Sequence Diagram 1

[link](#)





III. System Architecture UML Sequence Diagram 2

[link](#)

III. System Architecture - API Design

[google sheet link](#)

Status	Method	Name	URL (Edge Function)	Input	Output
ACTIVE	POST	Fetch Stocks	<code>supabase.co/function/s/v1/stocks</code>	<code>symbols: STR[] (e.g. ["KOSPI", "NASDAQ", "SP500", "USDKRW"])</code>	<code>{ symbol: STR, price: FLOAT, change: FLOAT, changePercent: FLOAT , ... }</code>
ACTIVE	POST	Fetch Trends	<code>supabase.co/function/s/v1/tavily</code>	<code>mode: "trends" query: STR</code>	<code>{ results: [{ title: STR, url: STR, content: STR, published_date: STR, image: STR}, ...], answer: STR }</code>
ACTIVE	POST	Fetch News	<code>supabase.co/function/s/v1/tavily</code>	<code>mode: "news" lat: FLOAT lon: FLOAT</code>	<code>{ results: [{ title: STR, url: STR, content: STR, published_date: STR, image: STR}, ... , answer: STR }</code>
ACTIVE	POST	Generate AI Brief	<code>supabase.co/function/s/v1/groq</code>	<code>prompt: STR system: STR model: STR</code>	<code>{ content: STR }</code>
ACTIVE	POST	Smart Widget Query	<code>supabase.co/function/s/v1/smart-widget</code>	<code>lat: FLOAT lon: FLOAT userInput: STR</code>	<code>{ keywords: STR[], suggestions: [...], weather: {...} }</code>

III. System Architecture - Deployment



Frontend Hosting: Vercel

- Automated build and deployment (CI/CD) via GitHub integration.
- Environment variable management for API Key security.
- Fast loading speeds and SSL/HTTPS by default.

Backend & Infrastructure: Supabase Cloud

- PostgreSQL: Managed database for real-time data processing.
- Edge Functions: Serverless runtime for data processing and AI communication.
- Auth: Management of secure user authentication systems such as Google OAuth.

Version Control: GitHub

- Management of entire source code and maintenance of branch strategies for team collaboration.

III. System Architecture - Code Conventions

Standard Frameworks:

- **Frontend (React/JS):** [Airbnb JavaScript Style Guide](#)
- **Backend (TypeScript):** [Google TypeScript Style Guide](#)

Naming Conventions:

- **Components/Classes:** `PascalCase` (e.g., `BriefingWidget`)
- **Variables/Functions:** `camelCase` (e.g., `fetchWeatherData`)
- **Database/Files:** `snake_case` or `kebab-case` (e.g., `user_settings`, `api-service.js`)

Formatting & Tools:

- **Indentation:** 2 Spaces (Soft Tabs)
- **Linters/Formatter:** ESLint & Prettier
- **Comments:** single-line and inline descriptions (`//`), JSDoc for complex functions (`/** ... */`)

IV. Schedule

HIGH: functional requirement level

Medium: use-case level

5/11/2026

Project Milestone 1

✓ Data Pipeline & UI Refinement

Dahyun: Design detailed wireframes for Persona settings and Diary Cards.

Sungmin: Optimize PostgreSQL schema for activity logging; set up RLS policies.

HIGH

Minsoo: Implement Smart Widgets & Briefing sync.

5/13/2026

Project Milestone 2

✓ Core Logic & Multi-language

Medium

Dahyun: Implement i18n logic for full Korean/English compatibility across the web.

HIGH

Sungmin: Develop integration logic to reflect Diary Card responses into the main Diary.

Medium

Minsoo: Integrate 30-day Keyword Decay formula; ensure API stability and caching

5/20/2026

Project Milestone 3

✓ AI Synthesis & History Fetch

Dahyun: Implement modal portals for detail views and Persona feedback UI.

HIGH

Sungmin: Integrate an API for Desktop History fetching and data parsing.

HIGH

Minsoo: Bridge Persona settings with Groq API; implement API error/timeout handling.

5/27/2026

Project Milestone 4

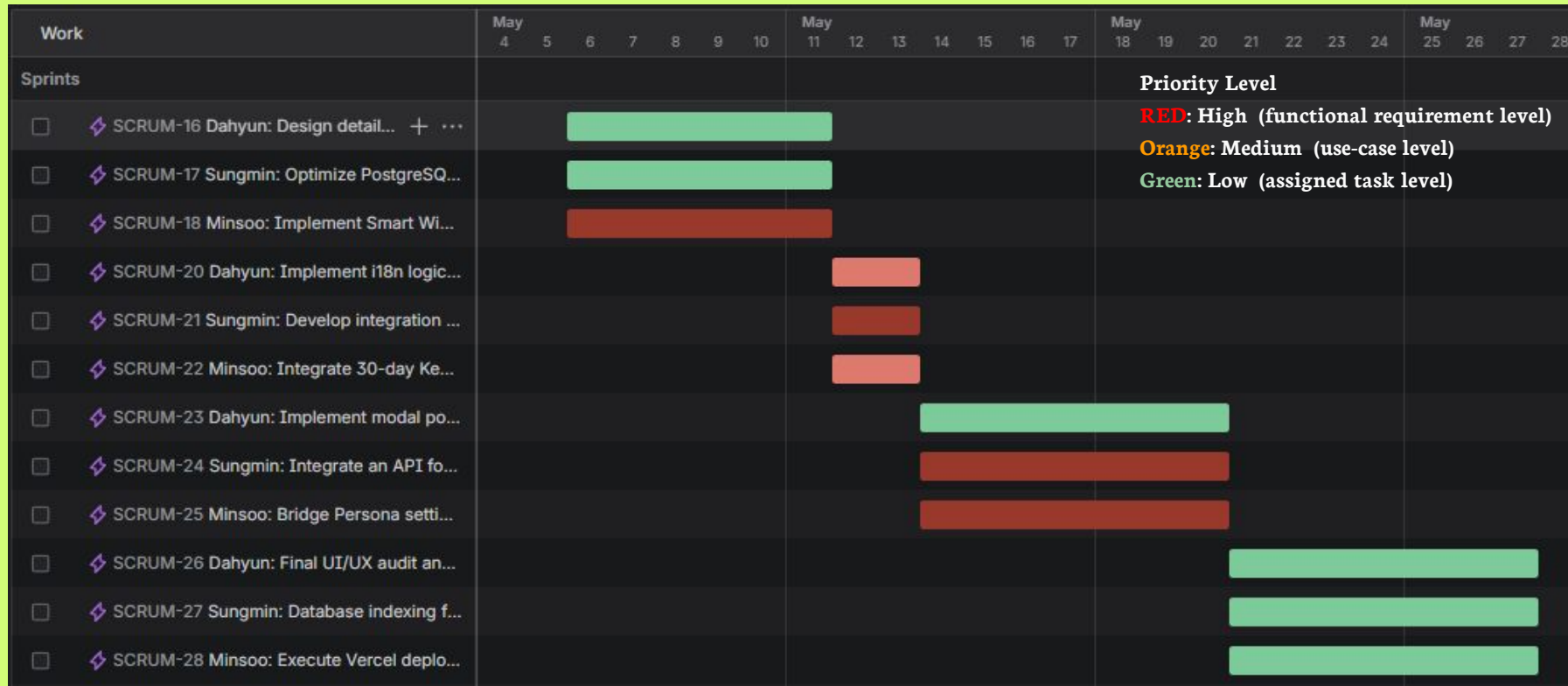
✓ Deployment & Maintenance

Dahyun: Final UI/UX audit and resolution of front-end design inconsistencies.

Sungmin: Database indexing for performance; finalize secure data isolation (RLS) audits.

Minsoo: Execute Vercel deployment; set up monitoring for API usage and maintenance.

IV. Schedule



Turn information into clarity.

Turn your day into a story.

Thank you.

Everything you need to know before your day begins in one screen.

AI summarizes your news, schedule, and tasks into a briefing built just for you.

When the day ends, AI writes your journal based on how your day actually went.

MorningBriefing.AI